

GamutX

File Handling

Python Basics



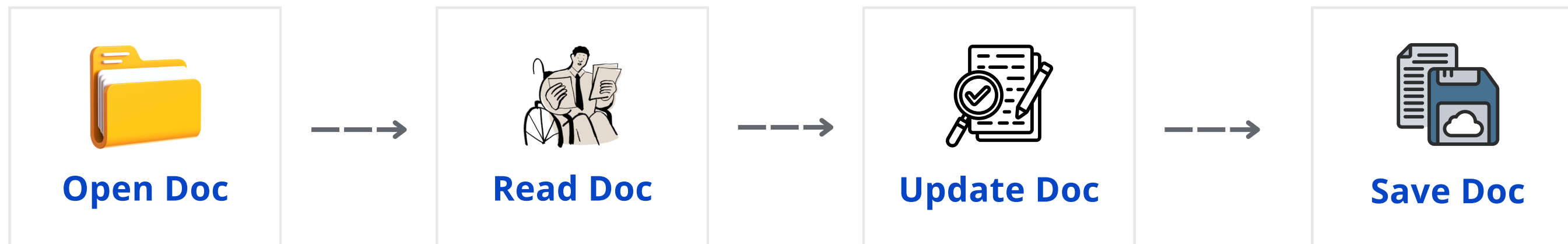
Introduction to File Handling

- What is File Handling?
- Using "r" Mode (Read Mode)
- Using "a" Mode (Append Mode)
- Using "w" Mode (Write Mode)
- Why Using open() Is Not the Best Practice

What is File Handling?

- File handling is the **process of creating, opening, reading, writing, and closing files in a program.**
- It allows us to **work with files** stored on our computer, such as **text files, images, or any other type of file.**

We can think of it like how you'd use a Word document



Python gives us tools to do all of this with a few simple commands.

File Modes

- **"r" - Read -**
 - Default value. Opens a file for reading, it returns an error if the file does not exist
- **"a" - Append -**
 - Opens a file for appending, creates the file if it does not exist
- **"w" - Write -**
 - Opens a file for writing, creates the file if it does not exist

Basic Syntax

```
file_object = open("filename", "mode")
```

how we want to use the file using
read, append or write mode

we provide file name ex. sample.txt

Using "r" Mode (Read Mode)

- This mode is used to read a file.
- If the file does not exist, an error will occur.

example.txt

```
this is line one  
this is line two  
this is line three  
this is line four  
this is line five
```

main.py

```
file = open("example.txt", "r")  
content = file.read()  
print(content)  
file.close()
```

```
# this is line one  
# this is line two  
# this is line three  
# this is line four  
# this is line five
```

Open file in read mode

used to read entire file

use to close file

Using "a" Mode (Append Mode)

- This is used to add content to the end of the file.
- If the file doesn't exist, Python will create it.

example.txt

```
this is line one  
this is line two  
this is line three  
this is line four  
this is line five
```

main.py

```
file = open("example.txt", "a")  
file.write("This line will be appended.\n")  
file.close()  
  
# this is line one  
# this is line two  
# this is line three  
# this is line four  
# this is line fiveThis line will be appended.
```

Open file in append mode

Add new content to the
end of file

✓ This won't remove existing content — it just adds to it.

Using "w" Mode (Write Mode)

- This is used to **write new content to a file**.
- If the **file exists**, it is **overwritten**.
- If the **file does not exist**, it is **created**.

example.txt

```
this is line one  
this is line two  
this is line three  
this is line four  
this is line five
```



main.py

```
file = open("example.txt", "w")  
  
file.write("This is new content written to the file.\n")  
file.close()
```

Open file in write mode

```
# This is new content written to the file.  
#
```

This will save in
example.txt file

⚠ **Be careful** — this will delete any existing content in the file.

Key Differences Between "r", "w", and "a":

- **"r" mode:**
 - Used for **reading** only.
 - The file must exist, or an error will occur.
- **"a" mode:**
 - Used for **appending** data to the end of the file.
 - If the file does not exist, it will be **created**.
- **"w" mode:**
 - Used for **writing** new data to a file.
 - If the file exists, it will be **overwritten**.
 - If the file does not exist, it will be **created**.

Why Using open() Is Not the Best Practice

- We have Seen How to work with file using open() Function
- This method works, but it comes with a risk.

```
file = open("example.txt", "r")  
content = file.read()
```

```
file.close()
```

Manual file closing required

! The Problem with Manual file.close() Lead to →

Resource Leaks

- The file stays open in memory, using system resources unnecessarily.

Inefficient Memory Usage

- Especially with large files or multiple open files, memory can fill up.

File Corruption

- If the program exits early, the file may not save correctly and could get corrupted.

Note: To solve this, Python gives us a better way using the with keyword.

Using with Statement

- What is with statement ?
- Reading r Mode
- Writing w Mode
- Appending a Mode

What is with statement ?

- The with statement in Python is a **cleaner and safer way to work with files**.
- When used, Python **automatically handles closing the file**
- This avoids common mistakes like forgetting to call file.close() manually.

Basic Syntax

```
with open("filename.txt", "r") as file:  
    # Perform file operations
```

Manages the file safely

Gives a name to the file object

Reading a File with with statement

```
# example.txt  
# This is sample text  
  
with open("example.txt", "r") as file:  
    content = file.read()  
    print(content)
```

```
# Output:  
# This is sample text
```

example.txt file
content

Reading r Mode

- Opens a file to read its contents.
- File **must exist** — **otherwise**, Python will **raise a FileNotFoundError**.

example.txt content

```
Hello, World!  
This is a test file.  
Have a nice day!
```



A dashed line with two arrows originates from the text box. One arrow points down to the 'Reading a File Line by Line' section, and the other points down to the 'Reading the Entire File' section.

Reading a File Line by Line

```
with open("example.txt", "r") as file:  
    for line in file: ←-----  
        print(line.strip())
```

```
# Hello, World!  
# This is a test file.  
# Have a nice day!
```

Reading the Entire File

```
with open("example.txt", "r") as file:  
    content = file.read()  
    print(content)
```

```
#Output:  
Hello, World!  
This is a test file.  
Have a nice day!
```

Writing w Mode

- In Python, we use "w" mode to write to a file.
 - If the file **already exists** — it gets **overwritten**.
 - If it **doesn't exist** — Python **creates a new file**.

Writing a Single Line

```
with open("output.txt", "w") as file:  
    file.write("Hello, World!")
```

```
# Hello, World!
```

- ◆ This writes **"Hello, World!"** to output.txt

Writing Multiple Lines

```
with open("output.txt", "w") as file:  
    file.write("First Line\n")  
    file.write("Second Line\n")  
    file.write("Third Line\n")
```

```
# First Line  
# Second Line  
# Third Line
```

- ◆ \n moves to the next line in the file. in output.txt

Appending a Mode

- the 'a' mode is used to append content to end to a file.
- **Does not delete existing content.**
- If the file doesn't exist, it is **created automatically.**

Output.txt

```
First Line  
Second Line  
Third Line
```



Appending a Single Line

```
with open("output.txt", "a") as file:  
    file.write("This line will be appended.\n")
```

```
# First Line  
# Second Line  
# Third Line  
# This line will be appended.
```

📌 This adds a new line to the end of output.txt

Concept Practice Task

Appending Multiple Lines to a File

```
# Example text file content: example.txt  
# Hello, World!  
# This is a test file.  
# Have a nice day!
```

```
with open("example.txt", "r") as file:  
    content = file.read()  
    print(content)
```

```
#Output:  
Hello, World!  
This is a test file.  
Have a nice day!
```

Various Reading Methods in Python

- Using - read() Method
- Using - readline() Method
- Using - readlines() Method
- read() vs readlines() vs readline()

Using - read() Method

- Reads the **entire content of a file at once**
- Stores everything as a **single string** — including newlines (\n) and spaces
- Useful when you want to process or **display the full content in one go**

File Content

Line 1
Line 2
Line 3



Code

```
with open("example.txt", "r") as file:  
    content = file.read()  
    print(content)
```

```
# Line 1  
# Line 2  
# Line 3
```

📌 When to Use

- The **file is small in size** (e.g., config file, small note).
- You need to **search, count, or work with the full text at once**.
- You're okay with the **file being read into one big string, including \n**.

Concept Practice Task

GamutX

Reading the Entire File

example.txt

```
Hello, this is an example file.  
It contains multiple lines of text.  
This is the last line.
```

Python Code

```
with open("example.txt", "r") as file:  
    content = file.read()  
    print(content)
```

Output:

```
Output:  
Hello, this is an example file.  
It contains multiple lines of text.  
This is the last line.
```

Reading an Empty File

```
with open("empty_file.txt", "w") as file:  
    pass # Just create an empty file  
  
with open("empty_file.txt", "r") as file:  
    content = file.read()  
    print("Content of the empty file:", content)
```

```
#Output:  
Content of the empty file:
```

Using - readline() Method

- **readline() reads only one line** from the file each time it is called.
- It remembers where it **left off**, so the next call reads the next line.
- It includes the **newline character \n** at the end (unless it's the last line)

File Content:

example.txt

Apple
Banana
Cherry

Code

```
with open("example.txt", "r") as file:  
    line1 = file.readline()  
    print("Line 1:", line1)  
  
    line2 = file.readline()  
    print("Line 2:", line2)  
  
# Line 1: Apple  
# Line 2: Banana
```

When to Use

- You want to **process or analyze lines one by one**.
- **Useful for large files** — it reads one line into memory at a time.

Concept Practice Task

Reading One Line at a Time

example.txt

```
Hello, this is line one.  
This is line two.  
And here is line three.
```

Python Code

```
with open("example.txt", "r") as file:  
    line1 = file.readline()  
    print("First Line:", line1)
```

Output:

```
First Line: Hello, this is line one.
```

Reading All Lines Using a Loop

```
with open("example.txt", "r") as file:  
    while True:  
        line = file.readline()  
        if not line:  
            break  
        print("Read Line:", line.strip())
```

Output:

```
Read Line: Hello, this is line one.  
Read Line: This is line two.  
Read Line: And here is line three.
```

Concept Practice Task

Reading Lines with a Specified Number of Lines

```
with open("example.txt", "r") as file:  
    for _ in range(2):  
        line = file.readline()  
        print("Read Line:", line.strip())
```

Output:

```
Read Line: Hello, this is line one.  
Read Line: This is line two.
```

Reading an Empty File with readline()

```
with open("empty_file.txt", "w") as file:  
    pass  
with open("empty_file.txt", "r") as file:  
    line = file.readline()  
    print("Content of the empty file:", line)
```

```
#Output:  
Content of the empty file:
```


Using - readlines() Method

- readlines() reads the whole file and returns a list of strings.
- Each line in the file becomes an item in the list.
- Each line string includes the \n character, unless it's the last line and has none.

File Content:

example.txt

Dog
Cat
Fish



Code

```
with open("example.txt", "r") as file:  
    lines = file.readlines()  
    for line in lines:  
        print("Line:", line.strip())
```

```
# Line: Dog  
# Line: Cat  
# Line: Fish
```

When to Use

- Need to store all lines for later use
- Want to count lines
- Access a specific line: lines[2]
- Great for searching or batch processing line by line

Concept Practice Task

Reading All Lines at Once

example.txt

```
Hello, this is line one.  
This is line two.  
And here is line three.
```

Python Code

```
with open("example.txt", "r") as file:  
    lines = file.readlines()  
    print(lines)
```

Output:

```
['Hello, this is line one.\n', 'This is line two.\n', 'And here is line three.\n']
```

Looping Over the List of Lines

```
with open("example.txt", "r") as file:  
    lines = file.readlines()
```

```
    for line in lines:  
        print(line.strip())
```

```
# output:  
Hello, this is line one.  
This is line two.  
And here is line three.
```


Concept Practice Task

GamutX

Reading a File with an Empty Line

example.txt

```
Hello, this is line one.  
  
This is line two.  
  
And here is line three.
```

Python Code

```
with open("example.txt", "r") as file:  
    lines = file.readlines()  
  
    for line in lines:  
        print(f"Line: {line.strip()}")
```

Output:

```
Line: Hello, this is line one.  
Line:  
Line: This is line two.  
Line:  
Line: And here is line three.
```


read() vs readlines() vs readline()

Method	Description	Returns	Use Case
• read()	→ Reads the entire content of the file as a single string.	→ A single string containing all the file's content.	→ When you need the entire content at once.
• readline()	→ Reads one line at a time from the file.	→ A single string (the next line).	→ When you want to process the file line by line.
• readlines()	→ Reads all lines from the file and returns them as a list.	→ A list of strings, each representing a line in the file.	→ At the beginning of functions, classes, or modules

Various Writing Methods

- Using write() Method
- Using the writelines() Method

Using write() Method

- The write() method lets you write text to a file in Python.
- You can write a single line or multiple lines —make sure to add \n if you want a new line.

Basic Syntax

```
file.write(string)
```



```
with open("example_write.txt", "w") as file:  
    file.write("This is the first line.\n")  
    file.write("This is the second line.\n")
```

```
# example_write.txt  
# This is the first line.  
# This is the second line.
```

📌 When to Use It

- We want to save some output to a file
- We're creating a log file or report
- We need to overwrite old content with new content

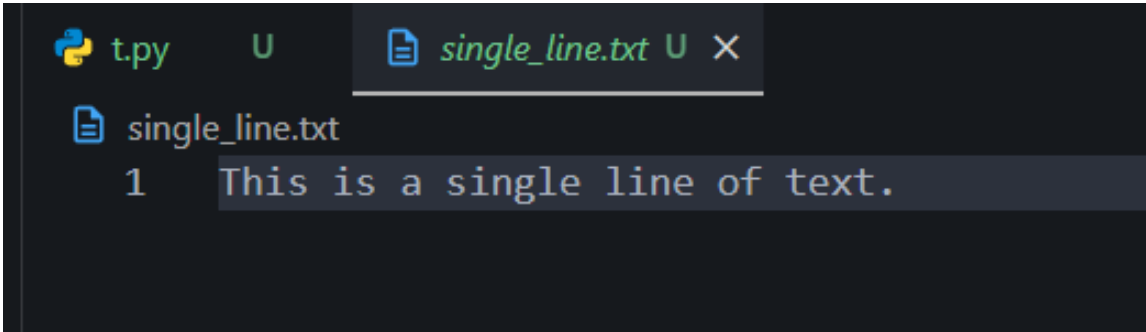
Concept Practice Task

GamutX

Writing a Single Line to a File

```
with open("single_line.txt", "w") as file:  
    file.write("This is a single line of text.")
```

Output:

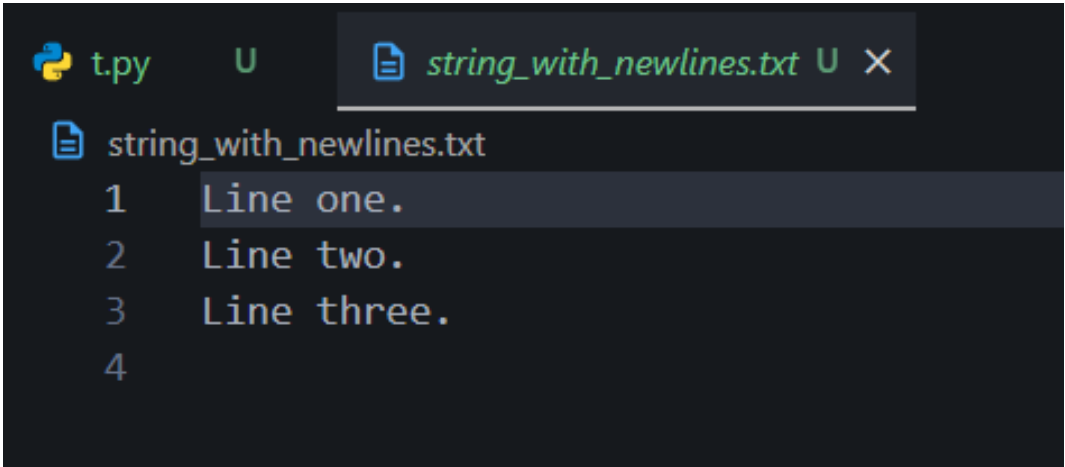


```
t.py  U  single_line.txt  U  X  
single_line.txt  
1  This is a single line of text.
```

Writing a String with Newlines

```
with open("string_with_newlines.txt", "w") as file:  
    file.write("Line one.\nLine two.\nLine three.\n")
```

Output:



```
t.py  U  string_with_newlines.txt  U  X  
string_with_newlines.txt  
1  Line one.  
2  Line two.  
3  Line three.  
4
```

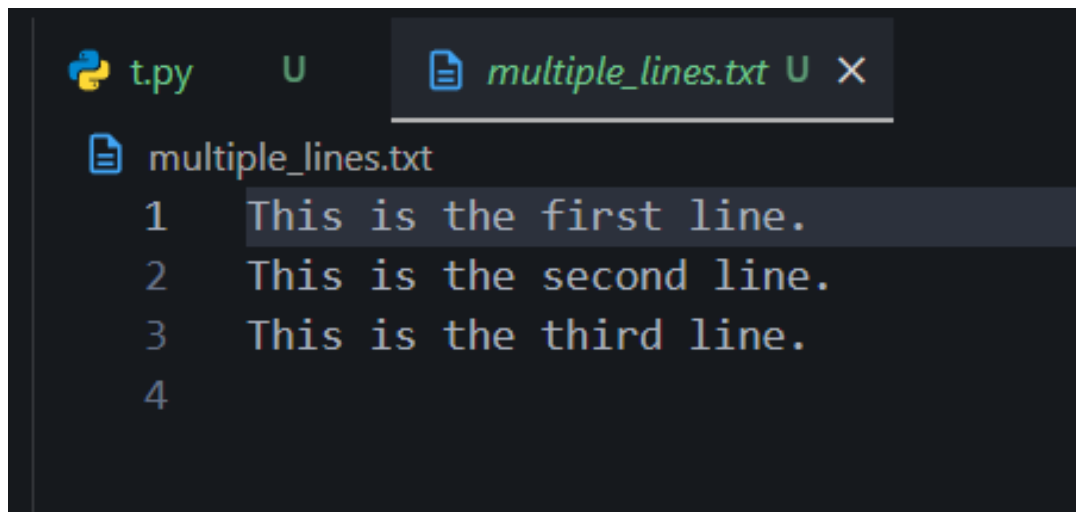
Concept Practice Task

GamutX

Writing Multiple Lines Manually

```
with open("multiple_lines.txt", "w") as file:  
    file.write("This is the first line.\n")  
    file.write("This is the second line.\n")  
    file.write("This is the third line.\n")
```

Output:

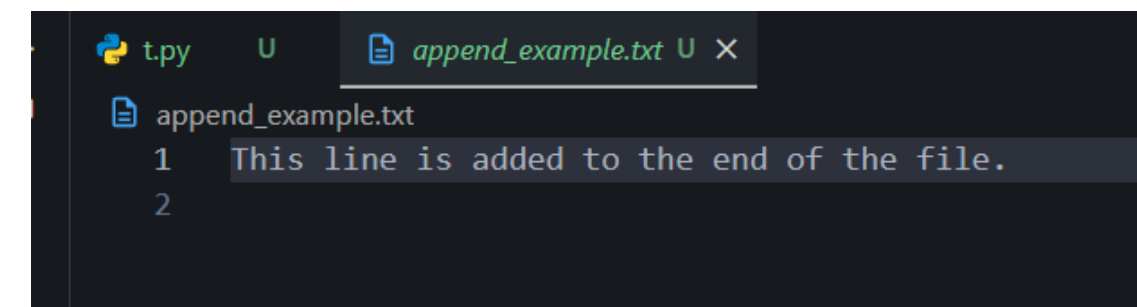


```
t.py  U  multiple_lines.txt U X  
multiple_lines.txt  
1 This is the first line.  
2 This is the second line.  
3 This is the third line.  
4
```

Appending to an Existing File

```
with open("append_example.txt", "a") as file:  
    file.write("This line is added to the end of the file.\n")
```

Output:



```
t.py  U  append_example.txt U X  
append_example.txt  
1 This line is added to the end of the file.  
2
```


Concept Practice Task

GamutX

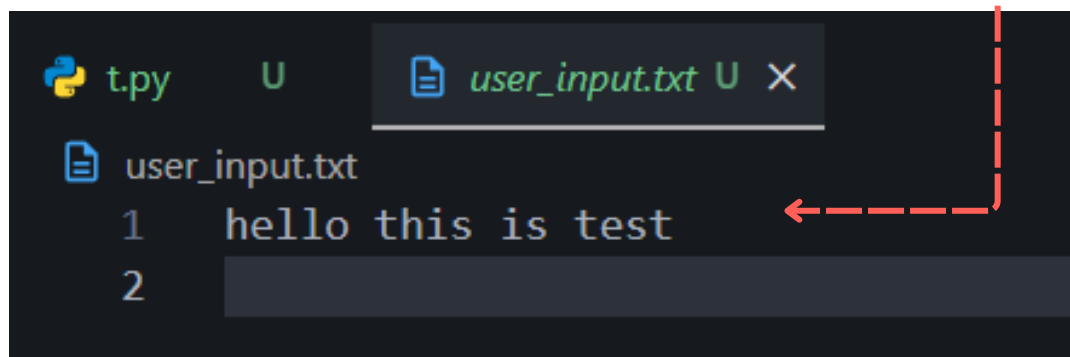
Writing User Input to a File

```
with open("user_input.txt", "w") as file:
    user_input = input("Enter a line of text to write to the file: ")
    file.write(user_input + "\n")
```

terminal

```
PS C:\Users\pralhad\Desktop\work\Academy\gamutx-VSD-MASTER> & C:/U
d/Desktop/Work/Academy/gamutx-VSD-MASTER/t.py
Enter a line of text to write to the file: hello this is test
PS C:\Users\pralhad\Desktop\work\Academy\gamutx-VSD-MASTER> |
```

Output:



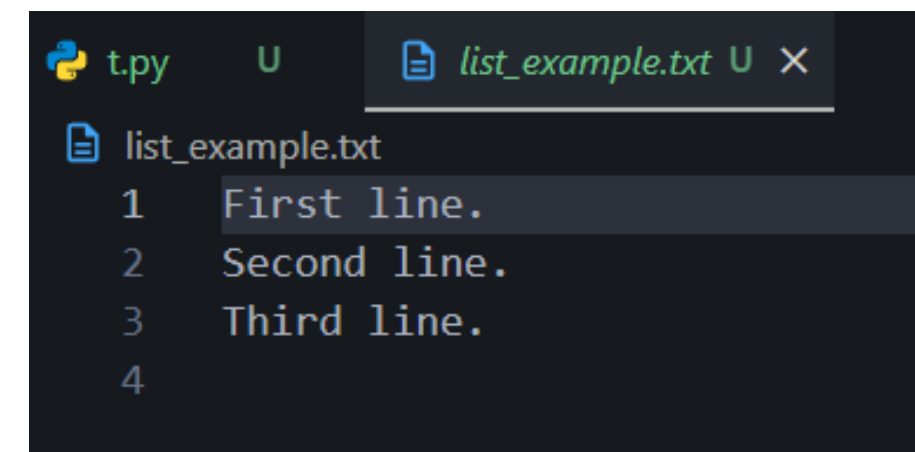
```
t.py U user_input.txt U X
user_input.txt
1 hello this is test
2
```

Writing a List of Strings (Manually)

```
lines = [
    "First line.\n",
    "Second line.\n",
    "Third line.\n"
]

with open("list_example.txt", "w") as file:
    for line in lines:
        file.write(line)
```

Output:



```
t.py U list_example.txt U X
list_example.txt
1 First line.
2 Second line.
3 Third line.
4
```

Using the writelines() Method

- Writes **multiple lines at once**.
- You must **provide a list (or other iterable) of strings**.
- Each string should already **include the newline character \n** if you **want lines to appear separately**.

Basic Syntax

```
file.writelines(iterable)
```

- **file:**
 - The file object you have opened for writing.
- **iterable:**
 - A collection of strings (like a list or tuple) that you want to write to the file.

Code Example

```
lines = [  
    "First line.\n",  
    "Second line.\n",  
    "Third line.\n"  
]  
  
with open("example_writelines.txt", "w") as file:  
    file.writelines(lines)
```

```
# example_writelines.txt  
# First line.  
# Second line.  
# Third line.  
#
```

Concept Practice Task

Writing User Input to a File

```
user_lines = []

for i in range(3):
    line = input(f"Enter line {i + 1}: ")
    user_lines.append(line + "\n")

with open("user_input.txt", "w") as file:
    file.writelines(user_lines)
```

terminal

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\pralhad\Desktop\Work\Academy\gamutx-VSD-MASTER> & C:/U
d/Desktop/work/Academy/gamutx-VSD-MASTER/t.py
Enter line 1: this is line 1
Enter line 2: this is line 2
Enter line 3: this is line 3
○ PS C:\Users\pralhad\Desktop\Work\Academy\gamutx-VSD-MASTER> |
```

Output:

```
t.py U user_input.txt U X
user_input.txt
1 this is line 1
2 this is line 2
3 this is line 3
4
```

Writing a List of Lines to a File

```
lines = [
    "First line.\n",
    "Second line.\n",
    "Third line.\n"
]

with open("lines_example.txt", "w") as file:
    file.writelines(lines)
```

Output:

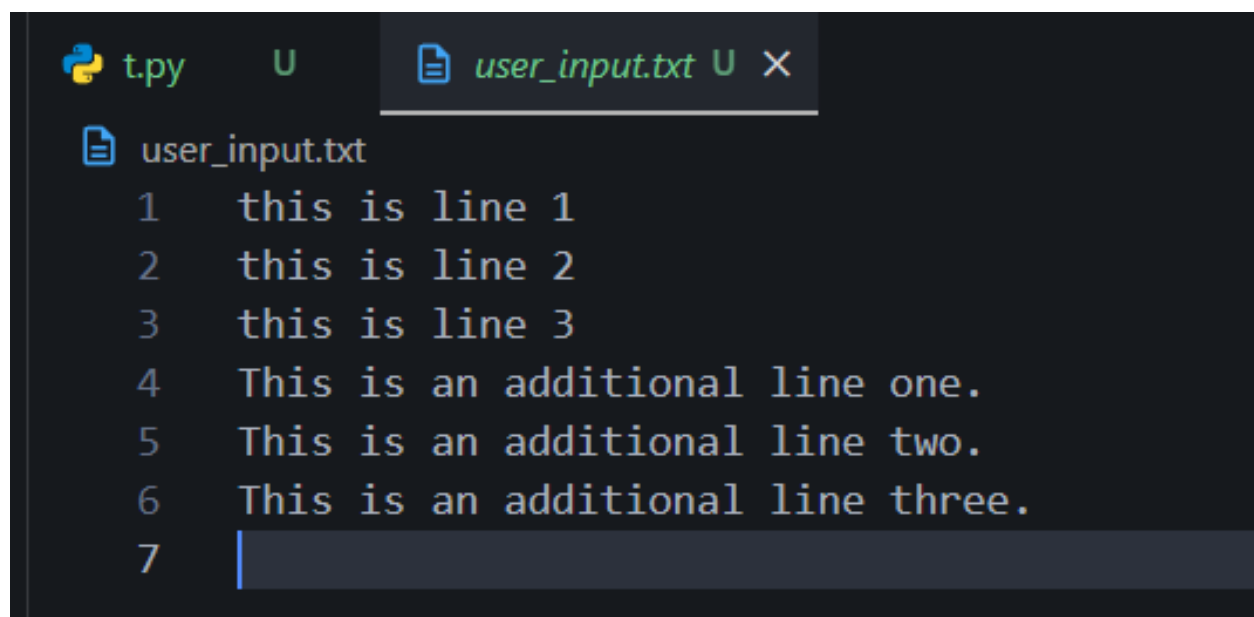
```
t.py U lines_example.txt U X
lines_example.txt
1 First line.
2 Second line.
3 Third line.
4
```


Concept Practice Task

Appending Multiple Lines to an Existing File

```
new_lines = [  
    "This is an additional line one.\n",  
    "This is an additional line two.\n",  
    "This is an additional line three.\n"  
]  
  
with open("user_input.txt", "a") as file:  
    file.writelines(new_lines)
```

Output:

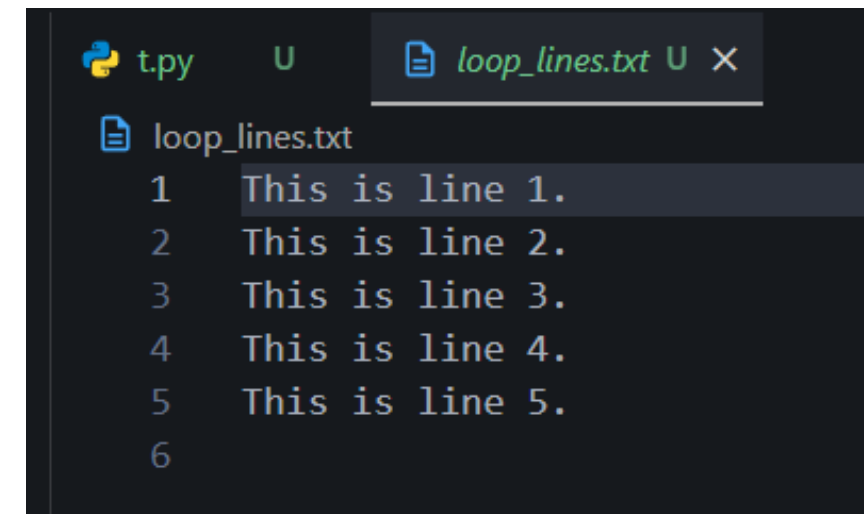


```
t.py  U  user_input.txt U X  
user_input.txt  
1  this is line 1  
2  this is line 2  
3  this is line 3  
4  This is an additional line one.  
5  This is an additional line two.  
6  This is an additional line three.  
7  
```

Writing Lines Using a Loop

```
lines = []  
  
for i in range(5):  
    lines.append(f"This is line {i + 1}.\n")  
  
with open("loop_lines.txt", "w") as file:  
    file.writelines(lines)
```

Output:



```
t.py  U  loop_lines.txt U X  
loop_lines.txt  
1  This is line 1.  
2  This is line 2.  
3  This is line 3.  
4  This is line 4.  
5  This is line 5.  
6  
```

Overwriting vs. Appending to Files

- Overwriting Files
- Appending to Files

Overwriting Files

- **Overwriting** = Replacing all content in a file with new content.
- Write mode ("w") does 3 things:
 - Deletes existing content in the file.
 - Writes the new content.
 - Creates the file if it doesn't exist.

main.py

```
# First write (creates the file)
with open("example.txt", "w") as file:
    file.write("This is the first line.\n")

# Second write (overwrites the previous content)
with open("example.txt", "w") as file:
    file.write("This is the second line.\n")
```

Output in example.txt

This **is** the second line.



Appending to Files

- Appending means **adding new content to the end of a file without deleting the existing data.**
- **How it works**
 - Keeps existing content
 - Adds new content at the end
 - **Creates** the file if it doesn't exist

main.py

```
# First append (adds new line)
with open("example.txt", "a") as file:
    file.write("This is an additional line.\n")
```

```
# Second append (adds another line)
with open("example.txt", "a") as file:
    file.write("This is another line.\n")
```

Output in example.txt

```
This is an additional line.
This is another line.
```

- The file keeps its existing content.
- Each `write()` adds more lines at the end.

Key Differences

Feature	Overwriting ("w")	Appending ("a")
• Existing Content	→ ✗ Erased	→ ✓ Preserved
• File Creation	→ Returns all keys as a view object	→ ✓ Yes (if not found)
• Cursor Starts At	→ Returns all values as a view object	→ End of the file
• Common Use Case	→ Returns key-value pairs as tuples	→ Add new data without removing old

📌 When to Use Which?

- **"w" (overwrite)** → we want to start fresh every time
- **"a" (append)** → we want to add new entries without losing existing ones

Common Mistakes in File Handling

- Forgetting to Close the File
- Incorrect File Mode
- File Not Found

Forgetting to Close the File

- When you open a file, you **need to close it after using it**.
- If you **don't close the file**:
 - Your **changes might not be saved**
 - It can **slow down your computer**
 - It **uses up system memory**

✗ Consequence

```
file = open("example.txt", "w")
file.write("Hello, World!")

# Forgetting to close the file
# file.close()
```

✓ Solution

```
with open("example.txt", "w") as file:
    file.write("Hello, World!")
```

- Always call `file.close()`, or better yet, use the **with statement**

Incorrect File Mode

- If you try to write to a file that's opened in read mode ("r"), Python will raise an error — because reading mode does not allow writing.

✗ Consequence

```
file = open("example.txt", "r")
file.write("This will cause an error!")

# Output
# Traceback (most recent call last):
#   File "c:\Users\HP\Desktop\nuke_tools\tool_loader.py", line 3, in <module>
#     file.write("This will cause an error!")
# io.UnsupportedOperation: not writable
```

Results in a **TypeError** since the file was not opened in write mode.

✓ Solution

Use the correct mode:

- Use **"w" for writing** (overwrites the file)
- Use **"a" for appending**
- Use **"r" if you want to read file**

File Not Found

- Trying to **open** a file in read mode ("r") when the file **doesn't exist** will raise a **FileNotFoundError**.

✗ Consequence

```
file = open("missing_file.txt", "r")

# Output
# Traceback (most recent call last):
#   File "c:\Users\HP\Desktop\nuke_tools\tool_loader.py", line 1, in <module>
#     file = open("missing_file.txt", "r")
# FileNotFoundError: [Errno 2] No such file or directory: 'missing_file.txt'
```

✓ Solution

```
try:
    with open("missing_file.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("The file does not exist.")
```

```
# The file does not exist.
```

- Make sure the **file actually exists** before reading.
- Use a **try-except block** to handle the error.

Thank You

